

Projeto da AV2: Modelo de Machine Learning Completo

Estrutura, Metodologia e Critérios de Entrega

Prof. Mateus Silva

Faculdade de Petrolina

2026

Por que um Projeto Prático?

- A teoria de ML é essencial, mas a prática consolida o aprendizado.
- Um projeto real exige **decisões técnicas**, **trade-offs** e **justificativas**.
- O objetivo: desenvolver um modelo ponta a ponta, da coleta ao deploy conceitual.

Filosofia do Projeto

Um modelo que funciona “perfeitamente” em testes mas não generaliza é inútil. Um modelo interpretável, robusto e validado tem valor real para o negócio.

Escopo do Projeto da AV2

O projeto deve cobrir **toda a cadeia** de um problema de ML real:

- 1 **Problemática:** definição clara do que será previsto/classificado.
- 2 **Dados:** coleta, limpeza, exploração e visualização.
- 3 **Preparação:** normalização, seleção de features, tratamento de missing/outliers.
- 4 **Modelagem:** escolha de algoritmos com justificativa.
- 5 **Validação:** CV, testes estatísticos e comparação de modelos.
- 6 **Interpretação:** insights acionáveis e limitações do modelo.
- 7 **Documentação:** relatório técnico e apresentação clara.

Pontuação e Pesos

Componente	Peso
Qualidade dos dados e preparação	15%
Seleção e justificativa de modelos	20%
Validação rigorosa (CV, testes, comparação)	25%
Interpretação e insights técnicos	20%
Documentação, código e apresentação	20%
Total	100%

Nota: Qualidade > Quantidade. Um modelo bem validado vale mais que 5 modelos ruins.

Regressão vs. Classificação: Escolha do Problema

Regressão

- Prever valor numérico contínuo.
- Métricas: MAE, RMSE, R^2 .
- Exemplos: preço de imóvel, demanda, temperatura.

Classificação

- Prever classe discreta (binária ou multiclasse).
- Métricas: Acurácia, F1, AUC-ROC.
- Exemplos: detecção de fraude, diagnóstico, churn.

Dica

Escolha um problema que **você entende** e para o qual **dados reais existem**. Evite problemas triviais ou impossíveis.

Exemplo 1: Regressão - Previsão de Preço de Imóvel

Problemática: Prever o preço de venda de imóveis em Petrolina com base em características.

Variável-alvo: Preço (contínuo, em R\$).

Features potenciais:

- Área (m^2), quartos, banheiros, garagem.
- Localização (bairro, distância centro).
- Idade do imóvel, tipo (casa/apartamento).
- Amenidades (piscina, ar-condicionado, etc).

Modelos apropriados: Regressão linear, Ridge, Lasso, SVR, Random Forest Regressor.

Métrica primária: RMSE (erro em R\$) ou MAE (interpretabilidade).

Desafio: dados faltantes em amenidades, outliers (imóveis de luxo muito diferentes).

Exemplo 2: Classificação Binária - Detecção de Fraude

Problemática: Detectar se uma transação bancária é fraude.

Classes: Fraude (1) vs. Legítima (0).

Features potenciais:

- Valor da transação, horário, localização geográfica.
- Histórico da conta (frequência, comportamento padrão).
- Tipo de transação (compra, saque, transferência).
- Dispositivo, IP, dados de risco externo.

Modelos apropriados: Logistic Regression, SVM, Random Forest, Naive Bayes.

Métrica primária: Precision-Recall (desbalanceamento severo: $\sim 0.1\%$ fraude).

Desafio: dados desbalanceados, custo assimétrico (falso negativo $>$ falso positivo).

Exemplo 3: Classificação Multiclasse - Classificação de Safra

Problemática: Classificar cultivares de tomate em estufa com base em características de planta.

Classes: Cultivar A, B, C, D.

Features potenciais:

- Altura, número de folhas, diâmetro do caule.
- Comprimento/largura de folhas, cor de fruto.
- Data de plantio, temperatura, umidade média.

Modelos apropriados: KNN, Logistic Regression (one-vs-rest), SVM multiclasse, Naive Bayes.

Métrica primária: F1 ponderado ou Macro-F1 (classes balanceadas).

Desafio: sobreposição de classes, captura de medidas precisas em campo.

Componente 1: Dados

Atividades esperadas:

- Coleta: buscar dados públicos ou sintéticos apropriados.
- Descrição: tamanho (n amostras $\times$ p features), distribuição das classes/valores alvo.
- Exploração (EDA): gráficos de distribuição, correlação, outliers, dados faltantes.
- Documentação: metadados claros (o que cada feature representa, unidades, range).

Entregável mínimo:

- Arquivo de dados (CSV, ou link público).
- Notebook ou script com EDA: ao menos 5 visualizações significativas.
- Descrição escrita de problemas encontrados (faltantes, outliers, desbalanceamento).

Componente 2: Preparação e Features

Atividades esperadas:

- Tratamento de dados faltantes (remoção, imputação, estratégia justificada).
- Tratamento de outliers (visual, IQR, Z-score, contexto de negócio).
- Normalização/Padronização (justificar a escolha por tipo de modelo).
- Seleção de features: manter as mais relevantes, remover redundância (correlação).
- Divisão treino/teste: proporção 70/30 ou 80/20 com random seed fixo.

Entregável mínimo:

- Código comentado com decisões: por que fazer assim?
- Antes/depois de tratamento: n amostras, distribuições.
- Matriz de correlação e seleção final de features com justificativa.

Componente 3: Modelagem

Atividades esperadas:

- Treinar **mínimo 3 modelos** diferentes (ex: Linear, SVM, KNN).
- Registrar hiperparâmetros e se usou valores padrão ou ajustes.
- Treinar na mesma divisão treino/teste para comparação justa.
- Não fazer ajuste massivo de hiperparâmetros no conjunto de teste (data leakage).

Entregável mínimo:

- Código de treinamento limpo e reproduzível.
- Tabela resumida: Modelo — Hiperparâmetros — Acurácia/RMSE Treino — Acurácia/RMSE Teste.
- Discussão inicial: qual modelo tem melhor desempenho?

Componente 4: Validação Rigorosa

Atividades esperadas:

- Validação cruzada (5-Fold ou 10-Fold) em treino para estimar desempenho futuro.
- Teste estatístico (McNemar, teste t pareado, Wilcoxon) para comparar modelos.
- Matriz de confusão e métricas por classe (se classificação).
- Curva ROC/AUC (se relevante para o problema).
- Análise de erros: que tipos de instâncias o modelo erra? Por quê?

Entregável mínimo:

- Resultados de CV com média e desvio padrão.
- Teste estatístico com p-value e conclusão.
- Gráfico de confusão e discussão de erros (falsos positivos, falsos negativos).

Componente 5: Interpretação e Insights

Atividades esperadas:

- Explicar quais features mais influenciam a predição (importância, coeficientes).
- Discutir limitações: dados insuficientes, viés, distribuição de treino $\neq$ realidade?
- Propor melhorias: mais dados, mais features, outro algoritmo?
- Conectar ao negócio: qual é o impacto prático do modelo?

Entregável mínimo:

- Gráfico de importância de features (se disponível para o modelo).
- Parágrafo escrito: 3–5 insights técnicos sobre o problema.
- Discussão de limitações e sugestões de melhoria.

Componente 6: Documentação e Apresentação

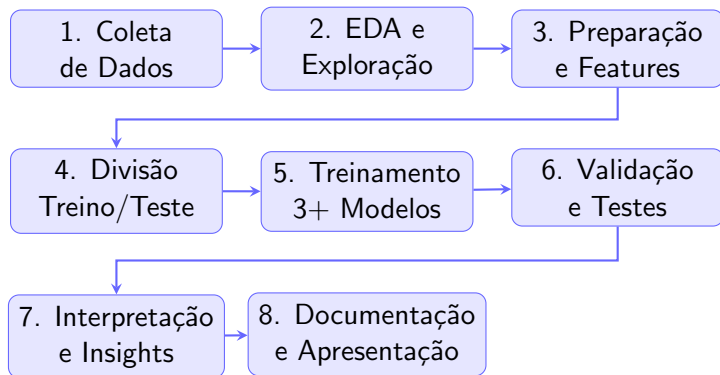
Atividades esperadas:

- Relatório técnico (2–4 páginas): problema, dados, modelos, resultados, conclusões.
- Código organizado, comentado e reproduzível (notebook ou .py + notebook de análise).
- Apresentação oral (10–15 min): resumo visual e discussão com professor.
- Responder perguntas sobre decisões técnicas e limitações.

Entregável mínimo:

- Relatório em PDF com seções: Introdução, Dados, Método, Resultados, Conclusão.
- Notebook executável que reproduz todo o pipeline.
- Apresentação com slides (mínimo 10 slides).

Fluxo Típico do Projeto



Nota: Esta não é uma sequência rígida. Pode haver iterações (ex: voltar a EDA após validar).

Checklist Obrigatório

Antes de entregar, verifique:

- 1 ✓ Dados descritos: tamanho, features, classe/alvo, problemas encontrados.
- 2 ✓ EDA com visualizações: ao menos 5 gráficos significativos.
- 3 ✓ Preparação documentada: tratamentos aplicados e justificativas.
- 4 ✓ Mínimo 3 modelos treinados e comparados.
- 5 ✓ Validação cruzada executada com resultados.
- 6 ✓ Teste estatístico entre melhores modelos com p-value.
- 7 ✓ Matriz de confusão (classificação) ou resíduos (regressão).
- 8 ✓ Análise de erros: por que o modelo falha?
- 9 ✓ Importância de features explicada.
- 10 ✓ Limitações discutidas, melhorias propostas.
- 11 ✓ Código reproduzível (notebook + requirements.txt).
- 12 ✓ Relatório técnico + apresentação pronta.

Critérios de Avaliação Detalhados

Qualidade dos Dados (15%)

- Dados apropriados e com quantidade suficiente ($n \geq 100$).
- EDA completa, problemas identificados e documentados.
- Tratamento de faltantes/outliers com critério técnico.

Modelagem (20%)

- Escolha de modelos justificada pelo tipo de problema.
- Hiperparâmetros documentados, não há data leakage.
- Múltiplos modelos treinados e comparados.

Critérios de Avaliação (Continuação)

Validação Rigorosa (25%)

- CV executada corretamente, resultados com média/desvio.
- Teste estatístico apropriado com interpretação.
- Métricas escolhidas de acordo com o contexto (não só acurácia).
- Análise de erros: compreensão de falhas do modelo.

Critérios de Avaliação (Continuação)

Interpretação (20%)

- Features mais relevantes identificadas e explicadas.
- Limitações reais discutidas (overfitting, viés, tamanho de amostra).
- Insights acionáveis para negócio/decisão.

Documentação (20%)

- Código limpo, bem-comentado e reproduzível.
- Relatório estruturado e claro.
- Apresentação profissional e responde às perguntas do professor.

Armadilha 1: Data Leakage

Problema: Normalizar com estatísticas de todo o dataset, incluindo teste.

Efeito: Métricas infladas, modelo pior na realidade.

Solução Correta

- 1 Dividir treino/teste **primeiro**.
- 2 Calcular estatísticas (média, desvio) **só no treino**.
- 3 Aplicar ao teste usando mesmos valores (não recalcular).

```
# ERRADO: normalizar antes de dividir
X = StandardScaler().fit_transform(X)
X_train, X_test = train_test_split(X)

# CERTO: dividir primeiro, depois normalizar
X_train, X_test = train_test_split(X)
scaler = StandardScaler().fit(X_train)
X_train = scaler.transform(X_train)
X_test = scaler.transform(X_test)
```

Armadilha 2: Ajustar Hiperparâmetros no Teste

Problema: Comparar múltiplos C (SVM) e K (KNN) escolhendo o melhor no teste.

Efeito: Overfitting aos dados de teste, desempenho real pior.

Solução Correta

- 1 Usar **Nested CV**: loop externo para estimar desempenho, interno para tuning.
- 2 Ou: ajustar hiperparâmetros em treino com CV, depois avaliar **uma única vez** em teste.

Regra de ouro: O teste é sagrado. Olhe uma única vez, no final.

Armadilha 3: Métrica Errada para o Problema

Problema: Usar acurácia em classificação desbalanceada (99% classe A, 1% classe B).

Efeito: Modelo que prevê sempre "A" tem 99% acurácia mas não funciona.

Solução Correta

- Classificação desbalanceada: F1, Precision-Recall, AUC-ROC.
- Regressão: MAE (interpretável), RMSE (penaliza outliers).
- Sempre usar a métrica apropriada ao **negócio**.

Armadilha 4: Não Documentar Decisões

Problema: Código sem comentários, ninguém entende as escolhas.

Efeito: Você esquece por que normalizou assim, qual foi o critério de features.

Solução Correta

- Comentar cada seção: *“Removemos X porque $\text{corr} > 0.95$ com Y ”*.
- Usar nomes claros: **X_train_scaled**, não **x1**.
- Manter arquivo de log: *“Testamos 50 hiperparâmetros, escolhemos $C=10$ por CV”*.

Armadilha 5: Confundir Modelo Perfeito com Modelo Bom

Problema: $F1 = 99\%$ no teste, confiança máxima, não testar generalização.

Efeito: Modelo falha miseravelmente em dados reais (distribuição diferente).

Solução Correta

- Sempre fazer CV: se $CV = 99\%$ mas teste = 95% , há overfitting.
- Testar em dados “novos” se possível (outro período, outra região).
- Documentar limitações: tamanho de amostra pequeno? Distribuição enviesada?
- Um modelo 85% bem validado é mais valioso que 99% duvidoso.

Pergunta 1: Preparação de Dados

Enunciado:

Você está desenvolvendo um modelo para prever a probabilidade de churn em uma operadora de telefonia. O dataset tem 10.000 clientes, mas 95% deles **não fizeram churn** e 5% **fizeram churn**.

Qual afirmação é **falsa** em relação a este cenário?

- A. A acurácia isolada é uma métrica inadequada neste caso.
- B. Usar F1-Score ou AUC-ROC seria mais apropriado que acurácia.
- C. Um modelo que prevê sempre “não churn” tem 95% acurácia e é excelente.
- D. Técnicas de balanceamento (SMOTE, class weights) podem ajudar.

Pergunta 2: Validação

Enunciado:

Um colega treina um classificador SVM e reporta:

- Acurácia em treino: 97%
- Acurácia em teste: 79%
- Validação cruzada (5-Fold): $80\% \pm 8\%$

Qual conclusão é **correta**?

- A. O modelo tem bias alto, não aprendeu bem.
- B. O modelo tem variância alta, overfitting ao treino.
- C. O modelo está bem, a CV confirma desempenho de $\sim 80\%$.
- D. O teste deve estar com erro sistemático, ignorar resultados.

Pergunta 3: Features e Normalização

Enunciado:

Você normaliza seus dados **antes** de dividir em treino/teste e treina um KNN com $k=5$. Posteriormente, avalia no teste e obtém acurácia de 92%.

O que **definitivamente** aconteceu?

- A. Houve data leakage, a métrica é inflada.
- B. O modelo generalizou bem, resultado é confiável.
- C. KNN é insensível à ordem das operações, não importa.
- D. A validação cruzada resolveu o problema.

Pergunta 4: Interpretação

Enunciado:

Após treinar um modelo de regressão linear, você obtém coeficientes:

- Intercepto: 50.000
- Área (m^2): +3.000
- Quartos: +25.000
- Localização (dummy, 1=Centro): +100.000

Qual interpretação é **correta**?

- A. Uma propriedade no centro custa R\$ 100.000 a mais, tudo igual.
- B. Cada quarto adicional custa R\$ 25.000, independente da escala.
- C. A localização é 100.000 vezes mais importante que quartos.
- D. O modelo não é interpretável sem ver as escalas de features.

Pergunta 5: Escolha de Modelos

Enunciado:

Você testou:

- Logistic Regression: $F1 = 0,78$
- SVM (RBF): $F1 = 0,79$
- Random Forest: $F1 = 0,77$

Baseado **apenas** nesses números, qual é a ação correta?

- A. Escolher SVM por ter melhor F1.
- B. Escolher Logistic Regression por ser interpretável e F1 estar perto de SVM.
- C. Executar teste estatístico (McNemar) para saber se a diferença é significativa.
- D. Usar ensemble de todos os 3 modelos.

Resposta 1: Preparação de Dados

Resposta Correta: C

Justificativa:

- Um modelo que prevê sempre “não churn” (classe majoritária) tem 95% acurácia **sem nenhum valor preditivo**.
- Afirmar que é “excelente” (alternativa C) é **completamente falso** e perigoso para o negócio.
- Alternativas A, B, D são todas corretas: acurácia é inadequada, F1/AUC melhores, balanceamento ajuda.

Lição

Em desbalanceamento severo, qualquer classificador baseline precisa de validação rigorosa. Nunca use acurácia isolada neste contexto.

Resposta 2: Validação

Resposta Correta: B

Justificativa:

- **Diferença entre treino (97%) e teste (79%):** 18 pontos é uma lacuna grande.
- **Indicador de variância alta:** o modelo se ajustou muito bem aos dados de treino (memorizou) mas falha em dados novos.
- **CV confirma realidade:** $80\% \pm 8\%$ é compatível com teste = 79%, a CV acerta.
- **Bias** seria o oposto: treino e teste ambos baixos.

Lição

CV é seu aliado. Se $CV \neq \text{Teste}$, verifique se não há erro; se $CV \approx \text{Teste}$ mas ambos $\ll \text{Treino}$, é overfitting.

Resposta 3: Features e Normalização

Resposta Correta: A

Justificativa:

- Normalizar **antes** da divisão significa usar **estatísticas de todo o dataset** (incluindo teste).
- Teste vê “pré-processamento conhecedor do futuro”: vazamento de informação (**data leakage**).
- KNN é **especialmente sensível** a escalas, então o vazamento inflou a métrica.
- Desempenho real em produção (dados verdadeiramente novos) será inferior a 92%.

Lição

Toda pipeline de pré-processamento deve ser **encaixada (fit) apenas em treino**. Leakage é sutil mas fatal para confiabilidade.

Resposta 4: Interpretação

Resposta Correta: D (com ressalva para A)

Justificativa:

- **Alternativa A é parcialmente correta:** sim, cada +1 no dummy Centro = +R\$ 100.000.
- Mas **alternativa D é a melhor resposta:** precisamos saber as escalas de features.
- Se Quartos está em [0–5] e Área em [50–500], unidades diferentes impactam magnitude dos coeficientes.
- **Alternativa B é enganosa:** Quartos e Área têm relação não-linear (mais quartos → mais área).
- **Alternativa C é falsa:** magnitude não mede importância relativa sem normalização.

Lição

Sempre normalize features antes de regressão linear se quiser comparar importância por coeficiente

Resposta 5: Escolha de Modelos

Resposta Correta: C

Justificativa:

- Diferenças: SVM (0,79) vs LR (0,78) vs RF (0,77) são pequenas (0,01–0,02).
- Sem teste estatístico, não sabemos se são **significativas** ou apenas ruído amostral.
- **McNemar**: teste de hipóteses apropriado para comparar dois classificadores em mesmos dados de teste.
- Se $p\text{-value} > 0.05$, as diferenças não são significativas → escolher por **simplicidade/interpretabilidade**.

Lição

Diferenças numéricas pequenas exigem rigor estatístico. Não basta ver quem tem melhor F1. Simplicidade ganha em caso de empate estatístico.

Cronograma Sugerido

Semana	Atividade	Entregável
Semana 1	Escolher problema, buscar dados	Proposta (1 pág)
Semana 2–3	EDA, preparação	Notebook com EDA completo
Semana 3–4	Modelagem (3+ modelos)	Resultados preliminares
Semana 4–5	Validação, testes estatísticos	Código final, análise
Semana 5	Relatório e apresentação	Projeto completo

Este é um cronograma flexível; ajuste conforme seu ritmo.

Recursos Recomendados

- **Datasets públicos:**

- Kaggle.com (competitions, datasets).
- UCI Machine Learning Repository.
- Data.gov (dados governamentais).

- **Documentação:**

- scikit-learn.org (exemplos, guias).
- Pandas documentation (manipulação de dados).

- **Referências livro:** Géron (Hands-On), Hastie et al. (Statistical Learning).

Dúvidas Finais?

Importante

- Venha falar comigo **antes de começar o projeto** se a ideia for muito ambiciosa ou impossível.
- Pergunte durante o desenvolvimento: é melhor esclarecer dúvidas cedo.
- Qualidade > Quantidade: um modelo bem validado vale mais que 10 modelos ruins.

Sucesso no projeto!